LA(1)LR(0)

임기정

Abstract

This note fixes the LR(0) characteristic automaton, defines the one-symbol lookahead set attached to each completed LR(0) item, and derives the usual Read/Follow digraph computation. The proofs are written so that the essential claims appear as inclusions, fixed-point equations, and derivation chains.

Contents

1 The LR(0) Automaton

Definition 1 (LRA(G))**.** Let $G \triangleq (N, T, P, S)$ be a context-free grammar. Define:

(i)
$$G' = (N', T', P', S') \triangleq \big(N \uplus \{S'\},\ T \uplus \{\$\},\ P \cup \{[S' \to S\$]\},\ S'\big).$$

(ii)
$$V \triangleq N \uplus T, \qquad V' \triangleq N' \uplus T'.$$

(iii)
$$\beta \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma \quad \overset{\text{def}}{\Longleftrightarrow} \quad (\beta, \gamma) \in \{(\alpha A z, \alpha\omega z) \mid [A \to \omega] \in P',\ \alpha \in V'^*,\ z \in T'^*\}.$$

(iv)
$$I_{\mathrm{LR}(0)} \triangleq \{[A \to \alpha \cdot \beta] \mid [A \to \alpha\beta] \in P'\}.$$

(v) For $q \subseteq I_{\mathrm{LR}(0)}$,
$$\mathrm{Cl}(q) =_\mu q \cup \{[A \to \varepsilon \cdot \omega] \mid [A \to \omega] \in P',\ [B \to \beta \cdot A\gamma] \in \mathrm{Cl}(q)\}.$$

(vi)
$$q_0 \triangleq \mathrm{Cl}\big(\{[S' \to \varepsilon \cdot S\$]\}\big).$$

(vii) For $q \subseteq I_{\mathrm{LR}(0)}$ and $X \in V'$,
$$\mathrm{GOTO}(q, X) \triangleq \mathrm{Cl}\big(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in q\}\big).$$

(viii)
$$Q \triangleq \mathrm{PT} \setminus \{\emptyset\}, \qquad \mathrm{PT} =_\mu \{q_0\} \cup \{\mathrm{GOTO}(q, X) \mid q \in \mathrm{PT},\ X \in V'\}.$$

(ix) The path relation is defined by structural recursion on the input word:
$$\varepsilon : p \to q \overset{\text{def}}{\Longleftrightarrow} p \in Q \wedge p = q, \qquad X\alpha : p \to q \overset{\text{def}}{\Longleftrightarrow} p \in Q \wedge \mathrm{GOTO}(p, X) \in Q \wedge \alpha : \mathrm{GOTO}(p, X) \to q.$$

Thus a path blocks exactly when the next GOTO-state is empty; the relation is deterministic, not an additional least fixed point.

(x)
$$\mathbf{Config} \triangleq \{(\alpha : p \to q, z) \mid \alpha \in V'^*,\ p, q \in Q,\ z \in T'^*,\ \alpha : p \to q\}.$$

(xi)
$$\delta(q, X) \triangleq \mathrm{GOTO}(q, X) \qquad (q \in Q,\ X \in V').$$

(xii)
$$\mathbf{reduce}(q, t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q\} \qquad (q \in Q,\ t \in T').$$

(xiii) Let $\vdash \subseteq \mathbf{Config} \times \mathbf{Config}$ be generated by:
$$\frac{q'' = \delta(q', t) \quad q'' \in Q}{(\alpha : q \to q', tz) \vdash (\alpha t : q \to q'', z)}\ \text{Shift}$$
$$\frac{\alpha : q \to p \quad \omega : p \to q' \quad [A \to \omega] \in \mathbf{reduce}(q', t) \quad q'' = \delta(p, A) \quad q'' \in Q}{(\alpha\omega : q \to q', tz) \vdash (\alpha A : q \to q'', tz)}\ \text{Reduce}(A \to \omega)$$

(xiv)
$$q_f \triangleq \delta(\delta(q_0, S), \$).$$

(xv) The accepted language is defined as
$$L(\mathrm{LRA}(G)) \triangleq \{z \in T^* \mid (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon)\}.$$

For $c = (\alpha : p \to q, u) \in \mathbf{Config}$, define the unread input by

$$\mathrm{input}(c) \triangleq u.$$

The accepting configurations are exactly

$$\mathsf{Accept}(c) \overset{\text{def}}{\iff} c = (S\$: q_0 \to q_f, \varepsilon),$$

and an error configuration is a non-accepting run configuration with no Shift or Reduce successor under the table being executed. In a mechanized definition, the displayed stack path is proof-carrying: the constructors for Shift and Reduce above include the side conditions that their target states lie in Q, so their conclusions are again elements of **Config**.

For $c = (\alpha : p \to q, u) \in \mathbf{Config}$, set $\mathrm{Y}(c) \in V'^*$ as

$$\mathrm{Y}(c) \triangleq \alpha u.$$

Lemma 2 (One-step yield invariant). *Each parser step reverses zero or one rightmost grammar step:*

$$c \vdash c' \Longrightarrow \begin{cases} \mathrm{Y}(c') = \mathrm{Y}(c), & \text{if the step is Shift,} \\ \mathrm{Y}(c') \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mathrm{Y}(c), & \text{if the step is Reduce.} \end{cases} \tag{1}$$

Proof. Indeed,

$$\begin{array}{rccll} \text{Shift :} & \mathrm{Y}(\alpha : q \to q', tz) & = & \alpha tz & = \mathrm{Y}(\alpha t : q \to q'', z), \\ \text{Reduce :} & \mathrm{Y}(\alpha A : q \to q'', tz) & = & \alpha A tz & \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha\omega tz = \mathrm{Y}(\alpha\omega : q \to q', tz). \end{array}$$

□

Corollary 3 (Multi-step yield invariant). *For $c, c' \in \mathbf{Config}$,*

$$c \vdash^* c' \Longrightarrow \mathrm{Y}(c') \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mathrm{Y}(c). \tag{2}$$

Proof. Induct on the length of $c \vdash^* c'$ and use Lemma 2 at each parser step. □

Corollary 4 (LR(0) soundness).

$$L(\mathrm{LRA}(G)) \subseteq L(G).$$

Proof.

$$\begin{aligned} z \in L(\mathrm{LRA}(G)) &\Longrightarrow (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon) \\ &\overset{(2)}{\Longrightarrow} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$ \\ &\Longrightarrow S \Rightarrow^*_P z \\ &\Longrightarrow z \in L(G). \end{aligned}$$

The third implication deletes the endmarker. Write the rightmost derivation witnessing $S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$$ as

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_M = z\$.$$

Since S' occurs on no right-hand side of P', the only production mentioning S', namely $[S' \to S\$]$, is never applicable to a form derived from $S\$$; hence every step uses a production of $P \subseteq P'$. Moreover $\$ \notin V$ occurs on no right-hand side of P, and no production of P rewrites a terminal, so the rightmost symbol $\$$ of γ_0 is never touched:

$$\gamma_k = \delta_k\$, \qquad \delta_k \in V^* \quad (0 \le k \le M), \qquad \delta_0 = S, \quad \delta_M = z, \qquad \delta_k \Rightarrow_P \delta_{k+1}.$$

Stripping the common trailing $\$$ therefore yields $S = \delta_0 \Rightarrow_P \delta_1 \Rightarrow_P \cdots \Rightarrow_P \delta_M = z$, i.e. $S \Rightarrow^*_P z$. □

Fact 5 (Elimination of non-productive nonterminals). *Call $A \in N'$* productive *if $A \Rightarrow^*_{P'} w$ for some $w \in T'^*$, and* non-productive *otherwise. Removing every non-productive nonterminal together with all productions that mention one preserves the generated language.*

Proof. Extend productivity to strings: $\sigma \in V'^*$ is productive if $\sigma \Rightarrow^*_{P'} x$ for some $x \in T'^*$. A terminal derives only itself and a derivation of a string is the concatenation of derivations of its symbols, so

$$Y_1 \cdots Y_m \text{ productive} \iff \forall i.\ Y_i \text{ productive} \qquad (t \in T' \text{ productive}). \tag{3}$$

<u>Fixed-point characterization</u>. Let $\mathrm{Gen} \subseteq N'$ be the least fixed point

$$\mathrm{Gen} = \bigcup_{n \geq 0} \mathrm{Gen}_n, \quad \mathrm{Gen}_0 = \emptyset, \quad \mathrm{Gen}_{n+1} = \{A \in N' \mid \exists [A \to X_1 \cdots X_k] \in P'.\ \forall i.\ X_i \in T' \cup \mathrm{Gen}_n\}$$

(with $k = 0$, i.e. $[A \to \varepsilon] \in P'$, permitted). Then A is productive iff $A \in \mathrm{Gen}$:

$$\begin{aligned} A \in \mathrm{Gen}_{n+1} &\Longrightarrow [A \to X_1 \cdots X_k] \in P' \text{ with } X_i \in T' \cup \mathrm{Gen}_n \\ &\overset{\mathrm{IH}_n}{\Longrightarrow} A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow^*_{P'} x_1 \cdots x_k \in T'^*; \\ A \Rightarrow^{\ell}_{P'} w \in T'^* \ (\ell \geq 1) &\Longrightarrow A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow^{\ell-1}_{P'} w,\ X_i \Rightarrow^{\ell_i}_{P'} w_i,\ \ell_i < \ell \\ &\overset{\mathrm{IH}_\ell}{\Longrightarrow} X_i \in T' \cup \mathrm{Gen} \Longrightarrow A \in \mathrm{Gen}. \end{aligned}$$

Thus the non-productive nonterminals are exactly $N' \setminus \mathrm{Gen}$, a least fixed point.

<u>Language preservation</u>. Let $\widehat{G}$ delete $N' \setminus \mathrm{Gen}$ and every production mentioning it; since terminals are productive, its productions are

$$\widehat{P} \triangleq \{[A \to X_1 \cdots X_k] \in P' \mid A, X_1, \ldots, X_k \text{ all productive}\} \subseteq P'.$$

As $\widehat{P} \subseteq P'$, $S' \Rightarrow^*_{\widehat{P}} z$ implies $S' \Rightarrow^*_{P'} z$. For the converse, induct on the length ℓ of $\sigma \Rightarrow^{\ell}_{P'} x \in T'^*$:

$$\sigma \Rightarrow^{\ell}_{P'} x \in T'^* \implies \sigma \text{ productive} \ \wedge\ \sigma \Rightarrow^*_{\widehat{P}} x.$$

For $\ell = 0$, $\sigma = x \in T'^*$. For $\ell \geq 1$, split off the first step $\sigma = \sigma_1 C \sigma_2$, $[C \to \gamma] \in P'$, $\sigma_1, \sigma_2 \in V'^*$:

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{P'} \sigma_1 \gamma \sigma_2 \Rightarrow^{\ell-1}_{P'} x \overset{\mathrm{IH}}{\Longrightarrow} \sigma_1 \gamma \sigma_2 \text{ productive} \ \wedge\ \sigma_1 \gamma \sigma_2 \Rightarrow^*_{\widehat{P}} x.$$

By (3), γ is productive, hence so is C (via $[C \to \gamma]$), so $[C \to \gamma] \in \widehat{P}$ and

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{\widehat{P}} \sigma_1 \gamma \sigma_2 \Rightarrow^*_{\widehat{P}} x.$$

Taking $\sigma = S'$ gives $L(G') \subseteq L(\widehat{G})$, so $L(\widehat{G}) = L(G')$ and every nonterminal of $\widehat{G}$ is productive.

When $L(G) \neq \emptyset$, S is productive, hence S' is productive ($[S' \to S\$]$) and $\widehat{G}$ keeps the start symbol with the same language. This is a language-level replacement only. It must not be used as a "without loss of generality" step for statements about the LR(0) automaton itself, because Cl, GOTO, Q, and all lookahead relations are rebuilt from the production set. Any theorem below that needs productivity states that hypothesis explicitly. ☐

The path relation factors through the state reached after any prefix: induction on $|\alpha|$ using Definition 1(ix) gives, for all $\alpha, \omega \in V'^*$ and $r \in Q$,

$$\alpha\omega : r \to q \iff \exists!\, p \in Q.\ \alpha : r \to p \ \wedge\ \omega : p \to q, \tag{4}$$

the intermediate state being the unique state obtained by following α from r, since each $\delta(\cdot, X) = \mathrm{GOTO}(\cdot, X)$ is single-valued.

Lemma 6 (Finite closed LR(0) states)**.** *The LR(0) construction satisfies the following structural invariants.*

(i) $I_{\mathrm{LR}(0)}$ *is finite, every* $q \in Q$ *is a subset of* $I_{\mathrm{LR}(0)}$*, and* Q *is finite.*

(ii) *Every* $q \in Q$ *is* Cl*-closed: if* $[B \to \beta \cdot A\gamma] \in q$ *and* $[A \to \omega] \in P'$*, then* $[A \to \varepsilon \cdot \omega] \in q$*.*

(iii) *If* $q \in Q$*,* $X \in V'$*, and* $\delta(q, X) \neq \emptyset$*, then* $\delta(q, X) \in Q$*. Equivalently, a nonempty target produced during the closure construction is one of the states of* Q*.*

Proof. The item set $I_{\mathrm{LR}(0)}$ is finite because P' is finite and each production has finitely many dot positions. The operator defining Cl only adds LR(0) items, so $\mathrm{Cl}(q) \subseteq I_{\mathrm{LR}(0)}$ whenever $q \subseteq I_{\mathrm{LR}(0)}$. Hence the fixed point PT lives inside the finite lattice $\mathcal{P}(I_{\mathrm{LR}(0)})$, and $Q = \mathrm{PT} \setminus \{\emptyset\}$ is finite. Closure-closedness is just unfolding the least fixed point defining Cl, and (iii) follows from how PT is closed under all GOTO successors. ☐

Lemma 7 (Derivation surgery used below). *The following context-free derivation facts are used as ordinary lemmas in the subsequent proofs.*

(i) Rightmostization for terminal targets. *If* $\sigma \Rightarrow^*_{P'} x \in T'^*$*, then* $\sigma \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} x$.

(ii) Terminal right-context insertion. *If* $\sigma \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \tau$ *and* $z \in T'^*$*, then* $\lambda\sigma z \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \lambda\tau z$ *for every left context* $\lambda \in V'^*$.

(iii) Terminal concatenation decomposition. *If* $\sigma_1\sigma_2 \Rightarrow^*_{P'} x \in T'^*$*, then* $x = x_1x_2$ *for some* $x_1, x_2 \in T'^*$ *with* $\sigma_i \Rightarrow^*_{P'} x_i$.

(iv) FIRST/path extraction. *Suppose a state* r *contains an item with the dot before a symbol* X*, and* $X \Rightarrow^*_{P'} t\chi$ *for* $t \in T'$*. Then, after following the nullable prefix of the leftmost spine witnessing this derivation, the LR(0) automaton has a nonempty* t*-transition. Equivalently, the terminal* t *obtained from* $\mathrm{FIRST}(X)$ *produces the* $\delta(\cdot, t) \neq \emptyset$ *premise used in Lemma 15.*

Proof. Items (i)–(iii) are the standard induction arguments on derivation length, using the fact that terminals are never rewritten. For (iv), take a leftmost spine from X to the first terminal t. The closure rule inserts the item for each nonterminal encountered on this spine, while the nullable siblings before the spine are followed by GOTO-steps. The first terminal item therefore gives a nonempty t-successor. In Coq these four statements are best made explicit once and then reused rather than reproved inside the main lookahead proof. ☐

Definition 8 (Viable prefixes and valid items). For $\eta \in V'^*$, say that η is a *viable prefix* of G' when it ends at some dot position of a handle in a right-sentential form:

$$\eta \text{ viable} \quad \overset{\text{def}}{\Longleftrightarrow} \quad \exists A, \alpha, \beta, \delta, w.\ [A \to \alpha\beta] \in P' \wedge \eta = \delta\alpha \wedge w \in T'^* \wedge S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta A w. \tag{5}$$

For the same η, define the valid LR(0) items by

$$[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta) \quad \overset{\text{def}}{\Longleftrightarrow} \quad [A \to \alpha\beta] \in P' \wedge \exists \delta, w.\ \eta = \delta\alpha \wedge w \in T'^* \wedge S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta A w. \tag{6}$$

Thus validity is just the item-indexed form of viability:

$$\mathsf{Valid}(\eta) \neq \emptyset \quad \Longleftrightarrow \quad \eta \text{ is a viable prefix of } G'. \tag{7}$$

Lemma 9 (One-sided LR(0) path invariants). *For arbitrary grammars, the characteristic automaton satisfies the directions needed for LR(0) completeness:*

$$\eta : q_0 \to q \Longrightarrow \mathsf{Valid}(\eta) \subseteq q, \tag{8}$$

$$\mathsf{Valid}(\eta) \neq \emptyset \Longrightarrow \exists q \in Q.\ \eta : q_0 \to q. \tag{9}$$

Moreover, every handle occurrence in a rightmost derivation is realized by a path and by the corresponding completed LR(0) item:

$$\begin{aligned} &S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta' A y \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \eta'\omega y, \qquad [A \to \omega] \in P', \qquad y = tz \in T'^* \\ &\quad \Longrightarrow \exists p, q \in Q.\ \eta' : q_0 \to p \wedge \omega : p \to q \\ &\qquad \wedge [A \to \omega \cdot \varepsilon] \in q \wedge [A \to \omega] \in \mathbf{reduce}(q, t). \end{aligned} \tag{10}$$

Proof. The key point is that no productivity assumption is needed for the inclusion $\mathsf{Valid}(\eta) \subseteq q$. Prove it simultaneously with the following closure-superset statement: if a kernel contains all valid items at η whose left part is nonempty, and contains the seed $[S' \to \varepsilon \cdot S\$]$ when $\eta = \varepsilon$, then its closure contains every item in $\mathsf{Valid}(\eta)$.

For a valid item with nonempty left part, the preceding dot position is valid at the previous prefix and is put into the predecessor state by induction; the GOTO-kernel then shifts the dot. For a valid item $[A \to \varepsilon \cdot \omega]$, choose the marked occurrence of A in the witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta A w$. If the derivation has length zero, the item is the augmented seed. Otherwise, the step that first introduces this marked occurrence has the form

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mu C \rho \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mu\sigma_1 A \sigma_2 \rho, \qquad [C \to \sigma_1 A \sigma_2] \in P', \qquad \eta = \mu\sigma_1,$$

and the rightmost-occurrence invariant says that the part to the left of the marked A is never changed after that step. The item $[C \to \sigma_1 \cdot A\sigma_2]$ is valid at η by the shorter witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mu C \rho$, hence is already in the kernel if $\sigma_1 \neq \varepsilon$, or is obtained by the induction hypothesis if $\sigma_1 = \varepsilon$. One closure step then adds $[A \to \varepsilon \cdot \omega]$. This proves (8) by induction on the path.

For (9), use (7) and induct on $|\eta|$. If $\eta = \varepsilon$, the seed makes $q_0 \neq \emptyset$ and $\varepsilon : q_0 \to q_0$. If $\eta = \eta_0 X$, a valid item at η is either itself a shifted item with left part ending in X, or the closure argument above traces it back to such a shifted valid item. By the induction hypothesis there is $p \in Q$ with $\eta_0 : q_0 \to p$, and (8) puts the predecessor item with dot before X in p. Hence $\mathrm{GOTO}(p, X) \neq \emptyset$, so it lies in Q by Lemma 6, giving the required path.

Finally, the premise of (10) makes $[A \to \omega \cdot \varepsilon]$ valid at $\eta'\omega$. By (9), choose $q \in Q$ with $\eta'\omega : q_0 \to q$, and factor this path uniquely by (4) as $\eta' : q_0 \to p$ and $\omega : p \to q$. Equation (8) puts the completed item in q, and Definition 1(xii) then gives $[A \to \omega] \in \mathbf{reduce}(q, t)$. ◻

Lemma 10 (LR(0) path exactness under productivity)**.** *Assume every nonterminal of G' is productive. Then every path state is exactly the corresponding valid-item set:*

$$\eta : q_0 \to q \Longrightarrow q = \mathsf{Valid}(\eta). \tag{11}$$

Consequently,

$$(\exists q \in Q)\ \eta : q_0 \to q \Longleftrightarrow \eta \text{ is a viable prefix of } G'. \tag{12}$$

Moreover, every path label is realized as the prefix of a right-sentential form:

$$\eta : q_0 \to q \Longrightarrow \exists z \in T'^*.\ S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \eta z. \tag{13}$$

Proof. The inclusion $\mathsf{Valid}(\eta) \subseteq q$ is (8). For the reverse inclusion, first prove closure soundness under productivity. Suppose a closure step adds $[A \to \varepsilon \cdot \omega]$ from $[B \to \beta_1 \cdot A\beta_2] \in \mathsf{Valid}(\eta)$, witnessed by $S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta B w$ and $\eta = \delta\beta_1$. Since every nonterminal is productive, the suffix β_2 derives some terminal string $x \in T'^*$; by Lemma 7, this can be done rightmost in the terminal right context. Hence

$$S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta B w \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \delta\beta_1 A \beta_2 w \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta\beta_1 A x w,$$

so $[A \to \varepsilon \cdot \omega] \in \mathsf{Valid}(\eta)$. Thus the closure of valid kernel items contains no invalid item. The GOTO step preserves validity by shifting the dot:

$$[A \to \alpha \cdot X\beta] \in \mathsf{Valid}(\eta) \Longleftrightarrow [A \to \alpha X \cdot \beta] \in \mathsf{Valid}(\eta X).$$

An induction on $|\eta|$ gives $q \subseteq \mathsf{Valid}(\eta)$ for every $\eta : q_0 \to q$, and therefore (11). Combining (11), (7), and the general viable-implies-path direction (9) proves (12).

For (13), let $\eta : q_0 \to q$. Since $q \in Q$, $q \neq \emptyset$; by (11), choose $[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta)$, witnessed by $\eta = \delta\alpha$, $S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta A w$, and $w \in T'^*$. Productivity gives $\beta \Rightarrow^*_{P'} x \in T'^*$, and Lemma 7 rightmostizes this derivation in the terminal right context:

$$S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta A w \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \delta\alpha\beta w \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta\alpha x w = \eta x w.$$

Thus $z = xw$ witnesses (13). ◻

Lemma 11 (LR(0) completeness)**.**

$$L(G) \subseteq L(\mathrm{LRA}(G)).$$

Proof. Let $z \in L(G)$, so $S \Rightarrow^*_P z$. Choose a rightmost derivation in G':

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \gamma_N = z\$. \tag{14}$$

Write its k-th step as

$$\gamma_k = \alpha_k A_k y_k \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \alpha_k \omega_k y_k = \gamma_{k+1}, \qquad [A_k \to \omega_k] \in P', \quad y_k = t_k z_k \in T'^*. \tag{15}$$

By (10), for each k there exist $p_k, q_k \in Q$ such that

$$\alpha_k : q_0 \to p_k, \qquad \omega_k : p_k \to q_k, \qquad [A_k \to \omega_k \cdot \varepsilon] \in q_k, \qquad [A_k \to \omega_k] \in \mathbf{reduce}(q_k, t_k). \tag{16}$$

Reading (14) from right to left turns each step into one inverse reduction. By (16) and (4), $\alpha_k\omega_k : q_0 \to q_k$, so the Reduce$(A_k \to \omega_k)$ rule applies:

$$(\alpha_k\omega_k : q_0 \to q_k, y_k) \vdash (\alpha_k A_k : q_0 \to \delta(p_k, A_k), y_k) \qquad (k = N-1, \ldots, 0). \tag{17}$$

These reductions are interleaved with Shift blocks. Since $y_k \in T'^*$, the symbol A_k exposed at step k is the rightmost nonterminal of $\gamma_k = \alpha_k A_k y_k$; using the previous step $\gamma_k = \alpha_{k-1}\omega_{k-1}y_{k-1}$ with $y_{k-1} \in T'^*$ and cancelling the common terminal suffix,

$$\alpha_k A_k\, u_k = \alpha_{k-1}\omega_{k-1}, \qquad y_k = u_k\, y_{k-1}, \qquad u_k \in T'^* \qquad (1 \le k \le N-1). \tag{18}$$

By (4) the path $\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1}$ factors through $\delta(p_k, A_k)$ after the prefix $\alpha_k A_k$, so the $|u_k|$ symbols of u_k are shifted one by one:

$$(\alpha_k A_k : q_0 \to \delta(p_k, A_k),\ u_k y_{k-1}) \ \vdash^{|u_k|}\ (\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1},\ y_{k-1}).$$

At the bottom ($k = N-1$), the run starts by shifting all of $\alpha_{N-1}\omega_{N-1}$ out of $z\$ = \gamma_N$:

$$(\varepsilon : q_0 \to q_0, z\$)\ \vdash^{|\alpha_{N-1}\omega_{N-1}|}\ (\alpha_{N-1}\omega_{N-1} : q_0 \to q_{N-1}, y_{N-1}).$$

At the top ($k = 0$) we have $\gamma_0 = S\$$, whence $\alpha_0 = \varepsilon$, $A_0 = S$, $y_0 = \$$; (17) leaves the stack S reading $\$$, and one final Shift of $\$$ gives

$$(S : q_0 \to \delta(q_0, S), \$)\ \vdash\ (S\$: q_0 \to q_f, \varepsilon), \qquad q_f = \delta(\delta(q_0, S), \$).$$

Concatenating the Shift blocks with (17) for $k = N-1, \ldots, 0$ gives

$$(\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon),$$

hence $z \in L(\mathrm{LRA}(G))$. ◻

Corollary 12 (Correctness of $\mathrm{LRA}(G)$).

$$L(G) = L(\mathrm{LRA}(G)), \qquad L(G) \triangleq \{z \in T^* \mid S \Rightarrow_P^* z\}.$$

Proof. Corollary 4 and Lemma 11 give

$$L(\mathrm{LRA}(G)) \subseteq L(G), \qquad L(G) \subseteq L(\mathrm{LRA}(G)).$$

◻

2 Read and Follow Sets

Define $\mathbf{LA} : \{(q, [A \to \omega]) \mid q \in Q,\ [A \to \omega \cdot \varepsilon] \in q\} \longrightarrow \mathcal{P}(T')$ by

$$\mathbf{LA}(q, [A \to \omega]) \triangleq \left\{ t \in T' \;\middle|\; S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z,\ \alpha\omega : q_0 \to q,\ \alpha \in V'^*,\ z \in T'^* \right\}. \tag{19}$$

The remainder of this section computes $\mathbf{LA}(q, [A \to \omega])$ from the LR(0) automaton, as the least fixed point of a Read/Follow digraph.

Let

$$D \triangleq \{(p, A) \mid p \in Q,\ A \in N',\ \delta(p, A) \neq \emptyset\}.$$

For a relation R, write

$$R\,!\,x \triangleq \{y \mid (x, y) \in R\}.$$

Define $\mathbf{Read}, \mathbf{Follow} \subseteq D \times T'$ as the least relations generated by the rules

$$\frac{\delta(\delta(p, A), t) \neq \emptyset}{t \in \mathbf{Read}\,!\,(p, A)}\ \mathrm{DR} \qquad\qquad \frac{\delta(p, A) = r \quad C \Rightarrow_{P'}^* \varepsilon \quad (r, C) \in D}{\mathbf{Read}\,!\,(r, C) \subseteq \mathbf{Read}\,!\,(p, A)}\ \text{reads}$$

$$\frac{t \in \mathbf{Read}\,!\,(p, A)}{t \in \mathbf{Follow}\,!\,(p, A)}\ \mathrm{Read} \qquad \frac{[B \to \beta \cdot A\gamma] \in p \quad \beta : p' \to p \quad \gamma \Rightarrow_{P'}^* \varepsilon \quad (p', B) \in D}{\mathbf{Follow}\,!\,(p', B) \subseteq \mathbf{Follow}\,!\,(p, A)}\ \text{includes}$$

Define the semantic read and follow sets on D:

$$\mathsf{Read}(p, A) \triangleq \{t \in T' \mid \exists r, s, \gamma.\ r = \delta(p, A) \land \gamma \Rightarrow_{P'}^* \varepsilon \land \gamma t : r \to s\}, \tag{20}$$

$$\mathsf{Follow}(p, A) \triangleq \left\{ t \in T' \;\middle|\; \exists \alpha, z.\ S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z \land \alpha : q_0 \to p \land z \in T'^* \right\}. \tag{21}$$

For $(p, A) \in D$, set

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Let

$$\Phi_R(F)(p, A) = \mathrm{DR}(p, A) \cup \bigcup_{\substack{\delta(p,A)=r \\ C \Rightarrow^*_{P'} \varepsilon \\ (r,C) \in D}} F(r, C).$$

Lemma 13 (Semantic Read fixed point)**.** *For $(p, A) \in D$,*

$$\mathbf{Read}\,!\,(p, A) = \mu\Phi_R(p, A) = \mathsf{Read}(p, A). \tag{22}$$

Proof. The two **Read**-rules say exactly that **Read**! is the least Φ_R-closed family, i.e. $\mathbf{Read}\,!\,(p, A) = \mu\Phi_R(p, A)$; since Φ_R is monotone on the finite lattice $D \to \mathcal{P}(T')$,

$$\mu\Phi_R = \bigcup_{m \geq 0} \Phi_R^m(\emptyset).$$

Write $(p, A) \rightsquigarrow_R (r, C)$ for the reads edge $r = \delta(p, A) \wedge C \Rightarrow^*_{P'} \varepsilon \wedge (r, C) \in D$. Because $\Phi_R(\emptyset)(p, A) = \mathrm{DR}(p, A)$, an induction on m unfolds the operator into reads-paths:

$$\Phi_R^{m+1}(\emptyset)(p, A) = \bigcup_{i=0}^{m} \bigcup_{(p,A) \rightsquigarrow^i_R (r,C)} \mathrm{DR}(r, C). \tag{23}$$

By (20), $t \in \mathsf{Read}(p, A)$ iff for some nullable $\gamma = C_1 \cdots C_n \in N'^*$,

$$r_0 = \delta(p, A), \qquad r_i = \delta(r_{i-1}, C_i)\ (1 \leq i \leq n), \qquad \delta(r_n, t) \neq \emptyset,$$

If $n = 0$, this says exactly that $t \in \mathrm{DR}(p, A)$. If $n > 0$, it is an n-edge reads-path ending at a DR-seed holding t:

$$(p, A) \rightsquigarrow_R (r_0, C_1) \rightsquigarrow_R \cdots \rightsquigarrow_R (r_{n-1}, C_n), \qquad t \in \mathrm{DR}(r_{n-1}, C_n) \quad \big(\delta(\delta(r_{n-1}, C_n), t) = \delta(r_n, t) \neq \emptyset\big).$$

In both cases (23) puts t in $\mu\Phi_R(p, A)$, and every member of the union (23) reads back as such a nullable δ-chain. Hence (22). ◻

Let

$$\Phi_F(F)(p, A) = \mathsf{Read}(p, A) \cup \bigcup_{\substack{[B \to \beta \cdot A\gamma] \in p \\ \beta : p' \to p \\ \gamma \Rightarrow^*_{P'} \varepsilon \\ (p',B) \in D}} F(p', B).$$

Lemma 14 (General Follow over-approximation)**.** *For arbitrary grammars and every $(p, A) \in D$,*

$$\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A) = \mathbf{Follow}\,!\,(p, A). \tag{24}$$

Proof. By Lemma 13, the two **Follow**-rules define the least fixed point of Φ_F. We prove $\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A)$ by induction on a marked rightmost derivation witnessing

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z, \qquad \alpha : q_0 \to p.$$

The occurrence of A cannot be the initial S' occurrence, because it is followed by the terminal t. Let the step that first introduces this marked occurrence be

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' B t' z' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow} \alpha' \beta A \gamma t' z' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z, \qquad \alpha = \alpha' \beta, \qquad [B \to \beta A \gamma] \in P'.$$

The marked-occurrence invariant for rightmost derivations says that, after this introducing step, the left context $\alpha'\beta$ and the marked A are never changed. Hence all remaining rewrites occur in the suffix descended from $\gamma t' z'$. Also $[B \to \beta \cdot A\gamma] \in \mathsf{Valid}(\alpha)$; since $\alpha : q_0 \to p$, the one-sided invariant (8) gives

$$[B \to \beta \cdot A\gamma] \in p. \tag{25}$$

Now split by the image of this particular γ-block in the displayed suffix derivation, not by the mere nullability of γ.

If the image of γ is nonempty, let t be its first terminal in the final suffix. Then $\gamma = \gamma_0 X \gamma_1$ where $\gamma_0 \Rightarrow^*_{P'} \varepsilon$ and $X \Rightarrow^*_{P'} t\chi$. Starting from (25), the state $\delta(p, A)$ contains the item $[B \to \beta A \cdot \gamma]$; following the nullable prefix γ_0 and using the FIRST/path extraction lemma gives a path $\gamma_0 t : \delta(p, A) \to s$ for some s. Thus $t \in \mathsf{Read}(p, A)$, and Lemma 13 puts t in the Read seed of $\Phi_F(p, A)$, hence in $\mu\Phi_F(p, A)$.

If the image of γ is empty, then $\gamma \Rightarrow^*_{P'} \varepsilon$ and the first terminal following the marked A is the old t', so $t = t'$. Factor the path $\alpha'\beta : q_0 \to p$ uniquely as $\alpha' : q_0 \to p'$ and $\beta : p' \to p$. The witness $S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' B t z$ is strictly shorter than the original marked witness, so the induction hypothesis gives $t \in \mu\Phi_F(p', B)$.

It remains to check that the includes edge is genuinely present. We already have (25), $\beta : p' \to p$, and $\gamma \Rightarrow^*_{P'} \varepsilon$. The fourth premise $(p', B) \in D$ follows from the same marked-occurrence argument. If $B = S'$, then the only S'-production is $S' \to S\$$, and the suffix γ in an item $S' \to \beta A \gamma$ with the dot before A contains \$, hence is not nullable. Thus $B \neq S'$. The live occurrence of B in $S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' B t z$ was therefore introduced by some production with the dot before B; that item is valid at α', and (8) puts it in p'. Hence $\delta(p', B) \neq \emptyset$, i.e. $(p', B) \in D$. The includes edge propagates t from (p', B) to (p, A), so $t \in \mu\Phi_F(p, A)$. ◻

Lemma 15 (Semantic Follow fixed point under productivity)**.** *Assume every nonterminal of G' is productive. For $(p, A) \in D$,*

$$\textbf{Follow}\,!\,(p, A) = \mu\Phi_F(p, A) = \mathsf{Follow}(p, A). \tag{26}$$

Proof. The inclusion $\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A)$ is Lemma 14. It remains to show that Follow is closed under Φ_F, for then the least fixed point is contained in Follow.

For a Read-seed, fix $t \in \mathsf{Read}(p, A)$. Since $(p, A) \in D$, the state p is reachable from q_0; choose α with $\alpha : q_0 \to p$. By (20), for some nullable γ there is a path $\gamma t : \delta(p, A) \to s$, hence $\alpha A \gamma t : q_0 \to s$. Productivity and Lemma 10 realize this path as a prefix of a right-sentential form: there exists $z \in T'^*$ such that

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A \gamma t z.$$

By Lemma 7, the nullable γ may be erased by rightmost steps in the terminal right context tz, giving

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A \gamma t z \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z.$$

Thus $t \in \mathsf{Follow}(p, A)$.

For an includes edge, assume

$$[B \to \beta \cdot A\gamma] \in p, \qquad \beta : p' \to p, \qquad \gamma \Rightarrow^*_{P'} \varepsilon, \qquad (p', B) \in D,$$

and take $t \in \mathsf{Follow}(p', B)$, witnessed by $S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' B t z$ and $\alpha' : q_0 \to p'$. The item supplies the production $B \to \beta A \gamma$, so

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' B t z \underset{\mathbf{rm}\ \ P'}{\Longrightarrow} \alpha' \beta A \gamma t z \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' \beta A t z,$$

where the last segment erases γ in the terminal right context. By (4), $\alpha'\beta : q_0 \to p$, hence $t \in \mathsf{Follow}(p, A)$. Therefore Follow is Φ_F-closed, and the least fixed point equality follows. ◻

Remark (Marked occurrences for a Coq formalization)**.** The informal phrase "the occurrence introduced by this production" is used in three places: the closure-superset direction of Lemma 9, the converse/over-approximation direction of Lemma 14, and the proof that the includes edge really has $(p', B) \in D$. In a formal development this should be represented once, either as a marked derivation relation or as a zipper over sentential forms.

The reusable invariant is simple: in a rightmost derivation, once a marked occurrence survives a step, the string strictly to its left is never changed by later steps. With this invariant, the introducing step for a marked A has a well-scoped factorization

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' B t' z' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow} \alpha' \beta A \gamma t' z' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha' \beta A t z,$$

and the two cases in Lemma 14 are determined by the actual image of the marked γ-block in this derivation segment. If that image is nonempty, the first terminal comes from a FIRST/path lemma and gives a Read seed. If that image is empty, then $\gamma \Rightarrow^* \varepsilon$, the terminal lookahead is inherited from the shorter B-witness, and the includes edge is available; the separate $(p', B) \in D$ premise follows by applying the same marked-occurrence invariant to the live occurrence of B. Thus the case split is not "γ nullable or not", but "the tracked γ-occurrence has empty image or nonempty image in this particular derivation".

Definition 16 (Digraph lookahead)**.** For every completed item $[A \to \omega \cdot \varepsilon] \in q$, define the lookahead set computed from the Read/Follow graph by

$$\widehat{\mathbf{LA}}(q, [A \to \omega]) \triangleq \{t \in T' \mid \exists p \in Q.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}\,!\,(p, A)\}. \tag{27}$$

Corollary 17 (Digraph lookahead is a safe over-approximation)**.** *For arbitrary grammars, whenever $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,*

$$\mathbf{LA}(q, [A \to \omega]) \subseteq \widehat{\mathbf{LA}}(q, [A \to \omega]). \tag{28}$$

Proof. Take $t \in \mathbf{LA}(q, [A \to \omega])$, witnessed by

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z, \qquad \alpha\omega : q_0 \to q.$$

By (4), factor the path as $\alpha : q_0 \to p$ and $\omega : p \to q$. Then $t \in \mathsf{Follow}(p, A)$ by (21). If $A = S'$, this situation cannot occur because S' occurs on no right-hand side and the initial occurrence is not followed by a terminal in any form derived from S'. Hence $A \neq S'$. Applying the marked-occurrence invariant to the live occurrence of A in $S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z$ yields a valid item at α with the dot before A; by (8) that item lies in p. Thus $\delta(p, A) \neq \emptyset$, so $(p, A) \in D$. Finally, Lemma 14 gives $t \in \mathbf{Follow}\,!\,(p, A)$, and the displayed definition of $\widehat{\mathbf{LA}}$ applies. ☐

Corollary 18 (Lookahead by Follow under productivity)**.** *Assume every nonterminal of G' is productive. Whenever $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,*

$$\mathbf{LA}(q, [A \to \omega]) = \widehat{\mathbf{LA}}(q, [A \to \omega]). \tag{29}$$

Proof. The inclusion $\mathbf{LA} \subseteq \widehat{\mathbf{LA}}$ is Corollary 17. Conversely, suppose $t \in \widehat{\mathbf{LA}}(q, [A \to \omega])$, witnessed by p with $\omega : p \to q$, $\delta(p, A) \neq \emptyset$, and $t \in \mathbf{Follow}\,!\,(p, A)$. By Lemma 15, $t \in \mathsf{Follow}(p, A)$; choose witnesses $S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha A t z$ and $\alpha : q_0 \to p$. Then (4) gives $\alpha\omega : q_0 \to q$, and the semantic definition (19) gives $t \in \mathbf{LA}(q, [A \to \omega])$. ☐

Theorem 19 (Digraph computation of lookahead)**.** *For arbitrary grammars, the Read/Follow digraph computes $\widehat{\mathbf{LA}}$, a safe over-approximation of the semantic lookahead $\mathbf{LA}$. If every nonterminal of G' is productive, then $\widehat{\mathbf{LA}} = \mathbf{LA}$ for every completed item.*

Proof. The Read fixed point is Lemma 13; the general semantic inclusion for Follow is Lemma 14, which gives Corollary 17. Under the explicit productivity hypothesis, Lemma 15 upgrades the inclusion to equality, and Corollary 18 gives the exact semantic formula. ☐

3 Lookahead Restriction

Recall the semantic lookahead set $\mathbf{LA}$ of (19). The same language-preservation argument below applies to any guard K satisfying

$$\mathbf{LA}(q, [A \to \omega]) \subseteq K(q, [A \to \omega]) \subseteq T'$$

for completed items, because soundness only uses $\mathbf{reduce}_K \subseteq \mathbf{reduce}$ and completeness only uses $\mathbf{LA} \subseteq K$. In particular, for arbitrary grammars one may take the digraph-computed over-approximation $K = \widehat{\mathbf{LA}}$ from Corollary 17; under productivity this is the exact semantic lookahead by Corollary 18. We state the formulas with the exact semantic guard $\mathbf{LA}$. Replace the LR(0) reduction function by

$$\mathbf{reduce}_{\mathbf{LA}}(q, t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q,\ t \in \mathbf{LA}(q, [A \to \omega])\}. \tag{30}$$

Let $\vdash_{\mathbf{LA}}$ be the transition relation obtained from $\vdash$ by replacing $\mathbf{reduce}$ with $\mathbf{reduce}_{\mathbf{LA}}$, and let $L(\vdash_{\mathbf{LA}})$ be its accepted language.

Lemma 20 (Guarded soundness)**.**

$$L(\vdash_{\mathbf{LA}}) \subseteq L(G).$$

Proof. From (30),

$$\mathbf{reduce}_{\mathbf{LA}}(q, t) \subseteq \mathbf{reduce}(q, t) \quad \Longrightarrow \quad \vdash_{\mathbf{LA}} \subseteq \vdash \quad \Longrightarrow \quad {\vdash_{\mathbf{LA}}}^* \subseteq \vdash^*.$$

Therefore

$$L(\vdash_{\mathbf{LA}}) \subseteq L(\mathrm{LRA}(G)) = L(G) \qquad \text{by Corollary 12.} \tag{31}$$

☐

Lemma 21 (Guarded completeness)**.**

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

Proof. Let $z \in L(G)$. In the accepting computation constructed in Lemma 11, every Reduce step has the form

$$(\alpha\omega : q_0 \to q, tz') \vdash (\alpha A : q_0 \to \delta(p, A), tz'),$$

where

$$S' \underset{\mathbf{rm}\ \ P'}{\Longrightarrow} S\$ \underset{\mathbf{rm}\ \ P'}{\Longrightarrow^*} \alpha Atz', \qquad \alpha\omega : q_0 \to q, \qquad [A \to \omega \cdot \varepsilon] \in q.$$

By (19),

$$t \in \mathbf{LA}(q, [A \to \omega]) \iff [A \to \omega] \in \mathbf{reduce_{LA}}(q, t).$$

Thus every Reduce step used in that accepting LR(0) computation is also legal for $\vdash_{\mathbf{LA}}$; Shift steps are unchanged. Hence

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

□

Corollary 22 (Language preservation)**.**

$$L(\vdash_{\mathbf{LA}}) = L(G).$$

Proof. Combine Lemmas 20 and 21. □

Definition 23 (LALR(1) action table)**.** For a chosen one-symbol guard K on completed items, the ACTION entries are

$$\mathsf{Act}_K(q, t) \triangleq \{\mathsf{shift}(\delta(q, t)) \mid \delta(q, t) \in Q\} \cup \{\mathsf{reduce}(A \to \omega) \mid [A \to \omega \cdot \varepsilon] \in q \wedge t \in K(q, [A \to \omega])\}.$$

A guarded LR(0) table is an LALR(1) table when

$$\forall q \in Q,\ t \in T'. \quad |\mathsf{Act}_K(q, t)| \le 1.$$

The associated parser is the transition system obtained by executing this single ACTION entry when it exists; if no ACTION entry exists and the configuration is not accepting, the configuration is an error configuration.

Lemma 24 (Conflict-free table condition)**.** *If the resulting table has no shift/reduce or reduce/reduce conflict, its action is single-valued under one-symbol lookahead, hence it is an* LALR(1) *table.*

Proof. At (q, t), the possible actions are

$$\delta(q, t) \quad \text{when this shift target is nonempty}, \qquad \mathbf{reduce_{LA}}(q, t).$$

Absence of shift/reduce and reduce/reduce conflicts means that this action is single-valued under one-symbol lookahead, which is precisely Definition 23. □

Theorem 25 (Lookahead-guarded reductions)**.** *If the guarded table is conflict-free, it is an* LALR(1) *parser accepting* $L(G)$*.*

Proof. Corollary 22 gives acceptance of $L(G)$, and Lemma 24 gives the LALR(1) table condition under the stated absence of conflicts. □

4 Language-Class Bridge

The productivity hypothesis in Theorem 19 is a hypothesis on a particular grammar and its LR(0) automaton. It is not a WLOG step inside an automaton-level proof. At the level of languages, however, the loss disappears: a semantic witness can be replaced by an equivalent productive certified witness. The replacement is existential; it rebuilds the grammar and therefore rebuilds the automaton.

Definition 26 (Semantic and certified witnesses)**.** A grammar G is a *semantic* LALR(1) *witness* when the ACTION table of Definition 23, built with the exact semantic guard $K = \mathbf{LA}_G$, is conflict-free. A grammar H is a *certified* LALR(1) *witness* when every nonterminal of H' is productive and the ACTION table built with the implemented guard $K = \widehat{\mathbf{LA}}_H$ is conflict-free. Define

$$\mathcal{L}_{\text{sem}} \triangleq \{L(G) \mid G \text{ is a semantic LALR(1) witness}\}, \qquad \mathcal{L}_{\text{cert}} \triangleq \{L(H) \mid H \text{ is a certified LALR(1) witness}\}.$$

The empty language is the only degenerate case excluded by the phrase "every nonterminal is productive": if $L(H) = \emptyset$, the start symbol of H is not productive. The theorem below therefore states the nonempty language-class claim. Empty input languages may be handled separately by an explicit reject-all recognizer, or by allowing the start symbol as a distinguished nonproductive exception.

Theorem 27 (Semantic witness to productive certified witness). *Let G be a grammar with $L(G) \neq \emptyset$. If G is a semantic* LALR(1) *witness, then there exists a grammar H, over the same terminal alphabet, such that*

$$L(H) = L(G), \qquad \textit{every nonterminal of } H' \textit{ is productive}, \qquad H \textit{ is a certified } \mathrm{LALR}(1) \textit{ witness.}$$

Consequently, on nonempty languages,

$$\mathcal{L}_{\text{sem}} = \mathcal{L}_{\text{cert}}.$$

Proof. We prove the nontrivial inclusion $\mathcal{L}_{\text{sem}} \subseteq \mathcal{L}_{\text{cert}}$. The reverse inclusion is immediate from Theorem 19: if H is productive, then $\widehat{\mathbf{LA}}_H = \mathbf{LA}_H$, so a certified conflict-free table is also a semantic conflict-free table.

1. Remove dead productions only at the language level. Let G° be the unaugmented productive core obtained by deleting from P every production whose left-hand side or right-hand side mentions a nonproductive nonterminal of G'. Since $L(G) \neq \emptyset$, the start symbol S is productive. Fact 5, with the trailing endmarker stripped as in the proof of Corollary 4, gives

$$L(G^\circ) = L(G).$$

This step is used only to know which productions can occur in terminal derivations. We do not claim that the LR(0) automaton of G may be replaced by the automaton of G° inside the previous proofs.

2. Split productive nonterminals by parser states. Let Q_G, δ_G, q_0 be the LR(0) automaton of the original grammar G', and let the conflict-free semantic ACTION table of G be built with $K = \mathbf{LA}_G$. We construct a finite split grammar $G^\bullet$. Its nonterminals are triples

$$\langle p, A, r \rangle \qquad (p, r \in Q_G,\ A \in N^\circ,\ \delta_G(p, A) = r),$$

where $N^\circ \subseteq N$ is the set of productive nonterminals of G', excluding the augmented symbol S'. The start symbol is

$$S^\bullet \triangleq \langle q_0, S, \delta_G(q_0, S) \rangle.$$

For every core production $A \to X_1 \cdots X_m \in G^\circ$ and every state sequence

$$p_0 \xrightarrow{X_1} p_1 \xrightarrow{X_2} \cdots \xrightarrow{X_m} p_m \quad \text{in } G,$$

meaning $p_i = \delta_G(p_{i-1}, X_i) \neq \emptyset$, add the production

$$\langle p_0, A, \delta_G(p_0, A) \rangle \to \Theta_1 \cdots \Theta_m,$$

where

$$\Theta_i = \begin{cases} X_i, & X_i \in T, \\ \langle p_{i-1}, X_i, p_i \rangle, & X_i \in N^\circ. \end{cases}$$

If $m = 0$, this is an ε-production for every pair $(p_0, \delta_G(p_0, A))$ with $\delta_G(p_0, A) \neq \emptyset$. Finally, let H be the productive core of $G^\bullet$, again by Fact 5. Because $L(G) \neq \emptyset$, the split start symbol is productive, so this trimming does not delete the start symbol. The augmented grammar H' is obtained from H in the usual way, by adding a fresh start production $S'_H \to S^\bullet\$$.

3. The split grammar is equivalent to the original grammar. Let π erase state annotations:

$$\pi(\langle p, A, r \rangle) = A, \qquad \pi(t) = t \quad (t \in T'), \qquad \pi(S'_H) = S'_G,$$

and extend π homomorphically to strings, productions, derivations, and parse trees. Every production of $G^\bullet$ erases to a production of the core G°, and the augmented production $S'_H \to S^\bullet\$$ erases to the original augmented production $S'_G \to S\$$. Hence every terminal derivation of $G^\bullet$, and therefore of H, erases to a terminal derivation of G°. Thus

$$L(H) \subseteq L(G^\circ).$$

Conversely, take $x \in L(G^\circ)$. Since G is a semantic witness, Corollary 22 gives an accepting run of the semantic lookahead parser for G on $x\$$. Read that run backwards as the rightmost derivation used in Lemma 11, ignoring only the final shift of \$. Each grammar reduction $A \to X_1 \cdots X_m$ pops a stack segment

$$p_0 \xrightarrow{X_1} p_1 \xrightarrow{X_2} \cdots \xrightarrow{X_m} p_m$$

and then pushes A, moving to $\delta_G(p_0, A)$. Replace this reduction by the corresponding split production displayed above, and annotate every child nonterminal X_i by the state pair (p_{i-1}, p_i). Induction over the reverse parse gives

$$S^\bullet \Rightarrow^*_{G^\bullet} x.$$

The derivation uses only productive split nonterminals, so it survives the final productive-core trimming. Therefore $x \in L(H)$, and

$$L(H) = L(G^\circ) = L(G).$$

By construction of the final core, every nonterminal of H is productive; and because $L(H) \neq \emptyset$, the augmented start symbol of H' is also productive via $S'_H \to S^\bullet\$$. Hence every nonterminal of H' is productive.

4. Split LR(0) states remember the original parser state. We record the invariant that prevents the productive trimming from reintroducing spurious LALR merging. For an item of H'

$$\langle p_0, A, r\rangle \to \Theta_1 \cdots \Theta_i \cdot \Theta_{i+1} \cdots \Theta_m,$$

define its *current original state* ρ to be the state obtained by following $\pi(\Theta_1 \cdots \Theta_i)$ from p_0 in the original automaton Q_G. This state is defined because the production was added only from a valid state sequence. The items of the augmented production $S'_H \to S^\bullet\$$ have no split left-hand side; assign them the current original states q_0, $\delta_G(q_0, S)$, and q_f at the three dot positions. This assignment is consistent with the reading below, since the dot before $S^\bullet = \langle q_0, S, \delta_G(q_0, S)\rangle$ has current state q_0; under $\pi(S'_H) = S'_G$ the three augmented items erase to $[S'_G \to \varepsilon \cdot S\$]$, $[S'_G \to S \cdot \$]$, and $[S'_G \to S\$ \cdot \varepsilon]$, which lie in q_0, $\delta_G(q_0, S)$, and q_f respectively. In particular the shift of \$ in the split automaton erases to the original shift of \$, so the conflict analysis below also covers the augmented items. Then every reachable LR(0) state $\bar{q}$ of H' has a unique state $\rho(\bar{q}) \in Q_G$ such that every item of $\bar{q}$ has current original state $\rho(\bar{q})$, and erasing annotations sends items of $\bar{q}$ to items of the original state $\rho(\bar{q})$.

The proof is by induction over the construction of reachable LR(0) states. Closure preserves the invariant: if the dot is before $\langle p, B, r\rangle$, then the current original state is exactly p, and all closure items for productions of $\langle p, B, r\rangle$ start in the same state p. GOTO preserves the invariant because shifting a terminal t or a split nonterminal $\langle p, B, r\rangle$ advances the current original state by $\delta_G(-, t)$ or $\delta_G(p, B) = r$, respectively. Erasing the moved item and then closing is exactly a subset of the corresponding original closure, since $G^\bullet$ uses only productions whose erasures are productions of G.

A useful consequence is uniqueness of split reductions inside one split state: if two completed split items in the same $\bar{q}$ erase to the same original production $A \to \omega$, then the split items are identical. First note that a split production is determined by its erasure together with its source state: the encoded state sequence is computed forward by $p_i = \delta_G(p_{i-1}, X_i)$, and the left-hand side is $\langle p_0, A, \delta_G(p_0, A)\rangle$. It therefore suffices to show that the two completed items carry the same source state p_0. If $\omega = \varepsilon$, both items have the dot at the left end; an item with the dot at the left end has current original state equal to its source state, so the invariant forces $p_0 = \rho(\bar{q})$ for both. If $|\omega| = m \geq 1$, both are kernel items of $\bar{q}$. Retreating the dot by one position is injective on items, and whenever $\mathrm{GOTO}(\bar{q}_1, \Theta) = \bar{q}$, every kernel item of $\bar{q}$ ends in Θ and its retreated item lies in $\bar{q}_1$; so both retreated items lie in every predecessor state of $\bar{q}$. Fix one access path of $\bar{q}$ from $\bar{q}_0$ and walk it backward m steps; this is possible because every item of $\bar{q}_0$ has the dot at the left end, so an access path of a state holding a kernel item at dot position m has at least m edges. After m retreats the two items lie, with the dot at the left end, in one common state $\bar{q}_{-m}$ on this path, and as in the $\omega = \varepsilon$ case the invariant gives $p_0 = \rho(\bar{q}_{-m})$ for both. Hence the two split items are identical. Backward determinism of δ_G is neither available nor used here: distinct source states may reach a common target under δ_G, and the forcing comes from the state invariant at a common predecessor state, not from inverting δ_G.

5. Semantic lookahead also erases. Let $\bar{I}$ be a completed item of H' in a split state $\bar{q}$, and let $\pi(\bar{I}) = [A \to \omega \cdot \varepsilon]$. If $t \in \mathbf{LA}_H(\bar{q}, \pi^{-1}(A \to \omega))$, then

$$t \in \mathbf{LA}_G(\rho(\bar{q}), [A \to \omega]).$$

To see this, take the semantic lookahead witness in H':

$$S'_H \Rightarrow^*_{H',\mathbf{rm}} \alpha\,\bar{A}\,tz, \qquad \alpha\bar{\omega} : \bar{q}_0 \to \bar{q}, \qquad z \in T'^*,$$

where $\bar{A}$ and $\bar{\omega}$ erase to A and ω. Erasing annotations gives

$$S'_G \Rightarrow^*_{G',\mathbf{rm}} \pi(\alpha)Atz, \qquad \pi(\alpha)\omega : q_0 \to \rho(\bar{q}),$$

by the state invariant. This is precisely the defining witness for $t \in \mathbf{LA}_G(\rho(\bar{q}), [A \to \omega])$.

6. Conflicts in the split grammar would be conflicts in the original table. Suppose the semantic ACTION table of H had a shift/reduce conflict at $(\bar{q}, t)$. The shift item erases, by the state invariant, to an original shift on t from $\rho(\bar{q})$. The reduce item erases, by the previous paragraph, to an original reduction whose semantic lookahead contains t at the same original state. This is a shift/reduce conflict in the semantic table of G, contradiction.

Similarly, suppose the semantic ACTION table of H had a reduce/reduce conflict at $(\bar{q}, t)$. By uniqueness of split reductions inside one split state, the two split reductions erase to two distinct original productions. By lookahead erasure, both original reductions are enabled under t at the same state $\rho(\bar{q})$. This is a reduce/reduce conflict in the semantic table of G, again a contradiction. Hence the semantic LALR(1) table of H is conflict-free.

Finally, every nonterminal of H' is productive, so Theorem 19 gives $\widehat{\mathbf{LA}}_H = \mathbf{LA}_H$. Therefore the implemented digraph guard and the semantic guard build the same ACTION table for H. Since the semantic table is conflict-free, the implemented table is conflict-free as well. Thus H is a productive certified LALR(1) witness for $L(G)$. ☐

Corollary 28 (Nonempty language-class equality)**.** *For every nonempty language $L \subseteq T^*$, the following are equivalent.*

1. $L \in \mathcal{L}_{\mathrm{sem}}$*: some grammar for L has a conflict-free semantic* LALR(1) *table.*
2. $L \in \mathcal{L}_{\mathrm{cert}}$*: some productive grammar for L has a conflict-free digraph-computed* LALR(1) *table.*

Proof. The implication (1) ⇒ (2) is Theorem 27. For (2) ⇒ (1), productivity gives $\widehat{\mathbf{LA}} = \mathbf{LA}$ by Theorem 19, so the certified table is the semantic table. ☐

A Digraph Algorithm and Implementation Specification

This appendix gives an executable specification for computing the sets used in Theorem 19. The construction is the usual digraph view of LALR(1) lookahead computation: direct seed sets are placed on nodes, dependency edges describe set inclusions, and the least fixed point is obtained by collapsing strongly connected components and propagating set unions along the resulting DAG [1, 2, 3].

A.1 General Digraph Fixed-Point Problem

Let X be a finite set of nodes, let $E \subseteq X \times X$ be a directed edge relation, and let $\mathrm{Seed} : X \to \mathcal{P}(T')$. We use the edge orientation

$$x \rightsquigarrow y \qquad \text{to mean} \qquad F(y) \subseteq F(x).$$

Equivalently, define the monotone operator

$$\Psi(F)(x) \triangleq \mathrm{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y).$$

The desired solution is the least fixed point

$$F = \mu\Psi, \qquad F(x) = \mathrm{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y). \tag{32}$$

Verified algorithm: work-list Kleene iteration. For formal verification, take $\mathrm{DIGRAPH}(X, E, \mathrm{Seed})$ to be the following finite work-list computation.

1. Initialize $F(x) \leftarrow \mathrm{Seed}(x)$ for all $x \in X$, and put all edges $x \rightsquigarrow y$ in a work-list.
2. While the work-list is nonempty, remove an edge $x \rightsquigarrow y$. If $F(y) \not\subseteq F(x)$, replace
$$F(x) \leftarrow F(x) \cup F(y)$$
and reinsert every incoming edge $u \rightsquigarrow x$.
3. Return F when the work-list is empty.

This is the implementation target used by the specification. It terminates because every update strictly increases some finite set $F(x) \subseteq T'$, so at most $|X| \cdot |T'|$ element insertions occur. The loop invariant is $\mathrm{Seed}(x) \subseteq F(x) \subseteq \mu\Psi(x)$ for all x. When the work-list is empty, every edge satisfies $F(y) \subseteq F(x)$, hence $\Psi(F) \subseteq F$; together with $\mathrm{Seed} \subseteq F$, this says F is Ψ-closed. Since $\mu\Psi$ is the least Ψ-closed family and the invariant gives $F \subseteq \mu\Psi$, the returned value is exactly $\mu\Psi$.

SCC optimization. The usual Tarjan-SCC/condensation-DAG algorithm computes the same fixed point by collapsing cycles and processing the resulting DAG in reverse topological order. It is a valid optimization once accompanied by a separate refinement proof that its result equals the work-list fixed point above. The mathematical development here relies only on the work-list correctness theorem, not on the internal correctness of a particular SCC implementation.

Complexity. Let $b = \lceil |T'|/w \rceil$, where w is the machine word size used for bitsets. The work-list implementation performs at most $|X| \cdot |T'|$ element insertions and scans incident edges when a set grows. With bitsets this is bounded by the usual

$$O\big((|X| + |E|)\, |T'|\, b\big)$$

worst-case work-list bound; the SCC implementation improves the practical bound to $O((|X| + |E|)b)$ after the SCC refinement proof is supplied.

A.2 Instance for Read

First compute nullable nonterminals by the least fixed point

$$\mathrm{Null}(A) \overset{\text{def}}{\iff} \exists [A \to X_1 \cdots X_k] \in P' \text{ such that every } X_i \text{ is nullable,}$$

where terminals are never nullable and $k = 0$ is allowed. Extend this to strings by

$$\mathrm{Null}(X_1 \cdots X_k) \overset{\text{def}}{\iff} \bigwedge_{i=1}^{k} \mathrm{Null}(X_i).$$

Lemma 29 (Null correctness). *For every $\gamma \in V'^*$,*

$$\mathrm{Null}(\gamma) \iff \gamma \Rightarrow^*_{P'} \varepsilon.$$

In particular, for $A \in N'$, $\mathrm{Null}(A)$ iff $A \Rightarrow^*_{P'} \varepsilon$.

Proof. The forward direction is induction on the stage at which a nonterminal enters the least fixed point, followed by concatenating the derivations of the symbols of γ. The reverse direction is induction on the length of a derivation to ε. A terminal cannot occur in such a derivation; for a first step $A \Rightarrow X_1 \cdots X_k \Rightarrow^* \varepsilon$, terminal concatenation decomposition gives $X_i \Rightarrow^* \varepsilon$ for each i, so the induction hypothesis makes every X_i nullable and the fixed-point rule adds A. The string case follows componentwise. □

For $x = (p, A) \in D$, define the direct-read seed

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Define $E_R \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_R (r, C) \overset{\text{def}}{\iff} \delta(p, A) = r \wedge C \in N' \wedge \mathrm{Null}(C) \wedge (r, C) \in D.$$

If $(r, C) \notin D$, then $\textbf{Read}\,!\,(r, C) = \emptyset$, so no edge is needed. Compute

$$\textbf{Read}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_R, \mathrm{DR})(p, A).$$

A.3 Instance for Follow

The seed for follow propagation is the already computed read set:

$$\mathrm{Seed}_F(p, A) \triangleq \textbf{Read}\,!\,(p, A).$$

Define $E_I \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_I (p', B) \overset{\text{def}}{\iff} [B \to \beta \cdot A\gamma] \in p \wedge \beta : p' \to p \wedge \mathrm{Null}(\gamma) \wedge (p', B) \in D.$$

The orientation means

$$\textbf{Follow}\,!\,(p', B) \subseteq \textbf{Follow}\,!\,(p, A).$$

Compute

$$\textbf{Follow}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_I, \mathrm{Seed}_F)(p, A).$$

A.4 Lookback Relation and Lookahead Construction

For every completed item $[A \to \omega \cdot \varepsilon] \in q$, define

$$\mathrm{LB}(q, A \to \omega) \triangleq \{(p, A) \in D \mid \omega : p \to q\}.$$

Then the implemented digraph guard is

$$\widehat{\mathbf{LA}}(q, [A \to \omega]) \triangleq \bigcup_{(p,A) \in \mathrm{LB}(q, A \to \omega)} \mathbf{Follow}\,!\,(p, A),$$

which is (27) written as an implementation step. It is always a safe over-approximation of the semantic **LA**, and it is exact under the productivity hypothesis of Corollary 18.

Implementation note. The query $\omega : p \to q$ can be implemented by following δ-edges from p along ω. The query $\beta : p' \to p$ in E_I can be implemented by caching reverse-path queries keyed by (β, p), or by storing predecessor edges during construction of the LR(0) automaton.

A.5 Parser-Table and Conflict Specification

After a guard K has been chosen—for instance $K = \widehat{\mathbf{LA}}$ or, under productivity, $K = \mathbf{LA}$—construct actions as follows.

1. If $\delta(q, t) \neq \emptyset$ for $t \in T'$, add the shift action
$$q \xrightarrow{t} \delta(q, t).$$
2. For each $[A \to \omega \cdot \varepsilon] \in q$ and each $t \in K(q, [A \to \omega])$, add the reduce action $A \to \omega$ under lookahead t.
3. Report a shift/reduce conflict when both a shift action and a reduce action are present for the same pair (q, t).
4. Report a reduce/reduce conflict when two distinct reduce productions are present for the same pair (q, t).

The accepting condition remains the one in Definition 1: the parser accepts when it reaches

$$(S\$: q_0 \to q_f, \varepsilon).$$

Equivalently, in a conventional ACTION/GOTO table, q_f may be treated as the accepting state after the input marker has been consumed.

A.6 Summary Specification

The complete computation required by Theorem 19 is:

1. Build G', Q, and δ as in Definition 1.
2. Compute nullable nonterminals and nullable strings.
3. Build D, DR, and E_R; compute **Read** by DIGRAPH(D, E_R, DR).
4. Build E_I; compute **Follow** by DIGRAPH$(D, E_I, \mathbf{Read})$.
5. Build LB and compute each $\widehat{\mathbf{LA}}(q, [A \to \omega])$ by unioning the appropriate **Follow**-sets.
6. Construct the guarded reduce actions and check conflicts.

The postcondition is that the computed **Read**, **Follow**, and $\widehat{\mathbf{LA}}$ are the least sets described by Theorem 19. For arbitrary grammars, $\widehat{\mathbf{LA}}$ safely over-approximates the semantic lookahead and therefore preserves the language when used as the guard in Section 3. Under the explicit productivity hypothesis, $\widehat{\mathbf{LA}} = \mathbf{LA}$, so the computed sets are exactly the semantic LALR(1) lookahead sets determined by the LR(0) automaton.

B Fuel-Free Parser Execution

The table specification above is relational. An executable parser can be obtained without adding a fuel argument exactly when no reachable reduce-only phase can run forever. This section states that condition using the actual parser stacks, not a finite control-only over-approximation.

Definition 30 (Run stacks and reachable phase heads)**.** A run stack is a viable stack path starting at the initial LR state:

$$\mathbf{Stack} \triangleq \{\alpha : q_0 \to q \mid \alpha \in V'^*,\ q \in Q,\ \alpha : q_0 \to q\}.$$

The corresponding run configurations are

$$\mathbf{Config}_{\mathrm{run}} \triangleq \{(s, u) \mid s \in \mathbf{Stack},\ u \in T'^*\}.$$

For an input word $w \in T^*$, put

$$c_0(w) \triangleq (\varepsilon : q_0 \to q_0, w\$).$$

A run configuration is reachable when

$$\mathsf{Reach}(c) \quad \overset{\text{def}}{\Longleftrightarrow} \quad \exists w \in T^*.\ c_0(w) \vdash^*_{\mathbf{LA}} c.$$

For $s \in \mathbf{Stack}$ and $t \in T'$, say that (s, t) is a reachable reduce-phase head when

$$\mathsf{Phase}(s, t) \quad \overset{\text{def}}{\Longleftrightarrow} \quad \exists u \in T'^*.\ \mathsf{Reach}(s, tu).$$

Definition 31 (Exact reduce-phase relation)**.** For each lookahead $t \in T'$, define a relation $\Rightarrow_t \subseteq \mathbf{Stack} \times \mathbf{Stack}$ by

$$s \Rightarrow_t s'$$

iff there exist $\alpha \in V'^*$, $A \in N'$, $\omega \in V'^*$, and states $p, q, q' \in Q$ such that

$$s = \big(\alpha\omega : q_0 \to q\big), \qquad s' = \big(\alpha A : q_0 \to q'\big),$$

$$\alpha : q_0 \to p, \qquad \omega : p \to q, \qquad [A \to \omega] \in \mathbf{reduce_{LA}}(q, t), \qquad q' = \delta(p, A).$$

Thus $s \Rightarrow_t s'$ records one actual guarded Reduce step with lookahead t, including the stack prefix that is exposed by popping the handle.

Lemma 32 (Exactness of reduce phases)**.** *For $s, s' \in \mathbf{Stack}$, $t \in T'$, and $u \in T'^*$, the following are equivalent:*

$$(s, tu) \vdash_{\mathbf{LA}} (s', tu) \quad \textit{by a Reduce step}$$

and

$$s \Rightarrow_t s'.$$

Consequently $\Rightarrow_t$ is neither forgetting the pop predecessor nor combining incompatible stack contexts.

Proof. Expanding the guarded Reduce rule, a Reduce step from (s, tu) to (s', tu) has the form

$$s = \big(\alpha\omega : q_0 \to q\big), \qquad s' = \big(\alpha A : q_0 \to q'\big),$$

with witnesses $p \in Q$, $A \in N'$, and $\omega \in V'^*$ satisfying

$$\alpha : q_0 \to p, \qquad \omega : p \to q, \qquad [A \to \omega] \in \mathbf{reduce_{LA}}(q, t), \qquad q' = \delta(p, A).$$

These are exactly the witnesses required by Definition 31. The converse is the same argument read backwards. □

Definition 33 (Exact reduce-phase termination)**.** For a relation R, write $\mathrm{Acc}_R(x)$ for accessibility of x: every R-successor of x is again accessible. The guarded table is *phase-terminating* when every reachable reduce-phase head is accessible for the exact reduce relation:

$$\forall s \in \mathbf{Stack},\ t \in T'.\quad \mathsf{Phase}(s, t) \Longrightarrow \mathrm{Acc}_{\Rightarrow_t}(s),$$

where $\mathrm{Acc}_{\Rightarrow_t}(s)$ uses the successor relation $s \Rightarrow_t s'$. This is the constructive well-foundedness formulation. The classically equivalent "there is no infinite reachable $\Rightarrow_t$-chain" formulation is not used as the definition.

Lemma 34 (Finite branching of exact reductions). *For each $s \in \mathbf{Stack}$ and $t \in T'$, the set*

$$\{s' \in \mathbf{Stack} \mid s \Rightarrow_t s'\}$$

is finite. If the guarded table is conflict-free, this set has at most one element.

Proof. Write $s = \beta : q_0 \to q$. A successor of s under $\Rightarrow_t$ must be produced by some completed item $[A \to \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q, t)$. This set of items is finite. For a fixed $A \to \omega$, the equality $\beta = \alpha\omega$ determines at most one prefix α, and the deterministic LR path from q_0 along α determines at most one exposed state p. Then the successor state is forced to be $\delta(p, A)$. Hence only finitely many successors can occur. If the table is conflict-free, at most one Reduce action is available at the pair (q, t), so there is at most one successor. ☐

Lemma 35 (Heights for finitely branching accessible relations). *Let $R \subseteq X \times X$ be finitely branching, with a finite successor list for each x. If $\mathrm{Acc}_R(x)$, then one can define, by well-founded recursion on this accessibility proof,*

$$H_R(x) \triangleq \begin{cases} 0, & \text{if } x \text{ has no } R\text{-successor,} \\ 1 + \max\{H_R(y) \mid x\ R\ y\}, & \text{otherwise.} \end{cases}$$

Moreover, if $x\ R\ y$, then $H_R(y) < H_R(x)$. Conversely, any map $H : X \to \mathbb{N}$ that strictly decreases along R gives $\mathrm{Acc}_R(x)$ for every x.

Proof. Accessibility gives the recursive calls $H_R(y)$ for every successor $x\ R\ y$; finite branching supplies the finite maximum. The definition therefore immediately yields $H_R(x) \geq 1 + H_R(y)$ for each successor, so $H_R(y) < H_R(x)$. Conversely, if H strictly decreases along R, prove $\mathrm{Acc}_R(x)$ by induction on $H(x)$: every successor has strictly smaller rank and is accessible by the induction hypothesis. ☐

Definition 36 (Exact termination certificate). For a conflict-free guarded table, a termination certificate is a rank function

$$\rho : \mathbf{Config}_{\mathrm{run}} \to \mathbb{N}$$

such that every reachable Shift or Reduce step strictly decreases

$$\mu(c) \triangleq (|\mathrm{input}(c)|, \rho(c)) \in \mathbb{N} \times \mathbb{N}$$

in lexicographic order. Concretely, whenever $\mathsf{Reach}(c)$ and $c \vdash_{\mathbf{LA}} c'$ is a Shift or Reduce step, one has

$$\mu(c') <_{\mathrm{lex}} \mu(c).$$

Accept and Error configurations make no recursive call.

Definition 37 (Recursive-call relation). Let $\mathsf{Call} \subseteq \mathbf{Config}_{\mathrm{run}} \times \mathbf{Config}_{\mathrm{run}}$ be the relation

$$c'\ \mathsf{Call}\ c$$

iff $\mathsf{Reach}(c)$, $c \vdash_{\mathbf{LA}} c'$, and that one parser step is either Shift or Reduce. Accept and Error configurations have no Call successors.

Lemma 38 (Certificate gives accessibility). *If a termination certificate ρ exists, then every reachable run configuration c satisfies*

$$\mathrm{Acc}_{\mathsf{Call}}(c).$$

Equivalently, the recursive parser may make recursive calls only to Call-smaller configurations.

Proof. Put $\mu(c) = (|\mathrm{input}(c)|, \rho(c))$. By Definition 36, every Call-edge $c'\ \mathsf{Call}\ c$ satisfies

$$\mu(c') <_{\mathrm{lex}} \mu(c).$$

The lexicographic order on $\mathbb{N} \times \mathbb{N}$ is well-founded, and well-foundedness is preserved by inverse image along μ. Hence the measure order

$$c' \prec_\rho c \quad \overset{\mathrm{def}}{\iff} \quad \mu(c') <_{\mathrm{lex}} \mu(c)$$

is well-founded on run configurations. Since Call is a subrelation of $\prec_\rho$ on reachable configurations, every reachable configuration is accessible for Call. ☐

Definition 39 (Canonical exact rank)**.** Assume the guarded table is phase-terminating. For $s \in \mathbf{Stack}$ and $t \in T'$, define $H_t(s)$ as follows. If $\mathsf{Phase}(s,t)$, use the accessibility proof supplied by Definition 33 and Lemma 35 for $\Rightarrow_t$. If $\neg\mathsf{Phase}(s,t)$, put $H_t(s) = 0$. For a run configuration $c = (s,u)$, put

$$\rho_{\mathrm{ex}}(c) \triangleq \begin{cases} H_t(s), & u = tu' \text{ for some } t \in T', u' \in T'^*, \\ 0, & u = \varepsilon. \end{cases}$$

This rank is a proof object: because Reach and Phase contain existential witnesses, ρ_{ex} is not by itself a complete decision procedure for arbitrary tables.

Theorem 40 (Sound and complete fuel-free criterion)**.** *For a conflict-free guarded table, the following implications hold constructively:*

(a) phase-termination in the sense of Definition 33 implies the existence of a termination certificate;

(b) a termination certificate implies that every public parser run starting from some $c_0(w)$, $w \in T^*$, is finite.

Conversely, in classical mathematics, finite public runs imply phase-termination; hence the three conditions are equivalent classically. When phase-termination holds, the canonical exact rank ρ_{ex} of Definition 39 is a termination certificate.

Proof. Phase-termination implies a certificate. Assume phase-termination. Let $c = (s,u)$ be reachable. If $u = tu'$, then $\mathsf{Phase}(s,t)$. By Definition 33, s is accessible for $\Rightarrow_t$; Lemma 34 and Lemma 35 give a height H_t that strictly decreases along every exact reduce edge.

We prove that ρ_{ex} is a termination certificate. If the recursive step from c to c' is a Shift and $\mathrm{input}(c) = tu'$, then $\mathrm{input}(c') = u'$, so

$$|\mathrm{input}(c')| < |\mathrm{input}(c)|.$$

Therefore $\mu(c') <_{\mathrm{lex}} \mu(c)$, independently of the second component. If the recursive step is a Reduce, write

$$c = (s, tu'), \qquad c' = (s', tu').$$

By Lemma 32, $s \Rightarrow_t s'$. The height decrease gives

$$\rho_{\mathrm{ex}}(c') = H_t(s') < H_t(s) = \rho_{\mathrm{ex}}(c).$$

The input length is unchanged by a Reduce step, so again $\mu(c') <_{\mathrm{lex}} \mu(c)$. Accept and Error configurations make no recursive call. Thus ρ_{ex} is a termination certificate.

A certificate implies finite public runs. Assume a termination certificate ρ exists. Along every reachable recursive parser step, the measure

$$\mu(c) = (|\mathrm{input}(c)|, \rho(c))$$

strictly decreases in the lexicographic order on $\mathbb{N} \times \mathbb{N}$. This order is well-founded, so no infinite sequence of reachable recursive calls can start from any $c_0(w)$. Hence every public parser run is finite.

Classical converse. Assume classical dependent choice, or equivalently the standard finitely branching form of Kőnig's lemma. If phase-termination failed, then some reachable phase head (s_0, t) would be non-accessible; finite branching would produce an infinite chain

$$s_0 \Rightarrow_t s_1 \Rightarrow_t s_2 \Rightarrow_t \cdots.$$

By $\mathsf{Phase}(s_0,t)$, choose u and w with $c_0(w) \vdash^*_{\mathbf{LA}} (s_0, tu)$. Lemma 32 turns each edge of the chain into a guarded Reduce step on the same unread input tu, so appending the infinite reduce phase to the finite prefix gives an infinite public parser run. Therefore, classically, finite public runs imply phase-termination. ☐

Corollary 41 (Fuel-free recursive recognizer)**.** *If the guarded table has a termination certificate, the public recognizer can be exposed without a fuel parameter:*

$$\mathrm{run\_parser} : T^* \to \mathrm{bool}.$$

It returns true *exactly for accepted inputs. A parser returning* option ParseTree *requires a separate tree-carrying configuration type and a proof that the produced tree denotes the corresponding derivation; that structure is not part of the recognizer defined in this note.*

Proof. Use the certificate rank to order recursive calls by

$$c' \prec c \quad \stackrel{\text{def}}{\iff} \quad \mu(c') <_{\text{lex}} \mu(c).$$

The lexicographic order on $\mathbb{N} \times \mathbb{N}$ is well-founded, and the certificate supplies the required decrease for every reachable Shift or Reduce call. Thus the internal runner is defined on reachable configurations by well-founded recursion on $\prec$, equivalently by carrying a reachability proof internally. The initial configuration $c_0(w)$ is reachable by the empty run, so the public wrapper supplies the accessibility proof and exposes no fuel argument. Its result is Boolean because the configurations above carry only control and input, not parse-tree fragments. □

The criterion above is exact as a mathematical specification: it rejects a table only when there is an actual infinite reduce-only phase reachable from some initial input. It is not, by itself, a complete executable decision procedure, because Reach, Phase, and the canonical rank ρ_{ex} contain existential proof data. A finite control-only graph may still be useful as a sufficient quick check, but it is not complete because such a graph can contain cycles that no single parser stack can realise. For an executable Coq development one should either use such sufficient checks or supply a table-specific termination certificate.

References

[1] F. DeRemer and T. J. Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4(4):615–649, 1982.

[2] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[3] J. H. Gallier. A survey of LR-parsing methods: The graph method for computing fixed points and LALR(1) lookahead sets. CIS 511 lecture notes, University of Pennsylvania, 2008.